DogCare

Plataforma móvil de servicios para el cuidado de perros

DOCUMENTACIÓN DE FIREBASE Y MODELO DE DATOS

Documentación de Firebase y Modelo de Datos

Proyecto	DogCare — Plataforma móvil de servicios para el cuidado de perros
Código del documento	DOC-04
Versión	1.0
Fecha	2026-11-02
Estándar de referencia	—
Institución	Universidad de San Buenaventura
Asignatura	Desarrollo de Aplicaciones para Dispositivos Móviles — Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Juan Esteban Olaya Muñoz
Nicolas Quiroz Gomez
Juan Sebastian Garzon Suarez

Tabla de contenido

1. Proyecto . . . 3

2. Authentication . . . 3

3. Colecciones y relaciones . . . 3

4. Reglas de seguridad (firestore.rules) . . . 4

5. Pruebas de las reglas . . . 4

6. Hallazgos de seguridad sobre las reglas . . . 4

7. Integración y configuración . . . 5

1. Proyecto

Elemento	Valor
Proyecto Firebase	`Jarvis` (ID `gen-lang-client-0991918834`, verificado con Firebase CLI en una sesión previa)
Base de datos Firestore	Base dedicada `ai-studio-android-dogcare-…` (no es `(default)`), declarada en `firebase.json`
Aplicación Android	`com.aistudio.dogcare.kprmdx`
Consola	Auth, usuarios, reglas desplegadas e índices: NO se pudieron inspeccionar en esta ejecución (servidor MCP de Firebase sin conexión); estado BLOQUEADO / PENDIENTE.

2. Authentication

Proveedores en el código: correo y contraseña, y Google (Credential Manager). El perfil se sincroniza en `users/{uid}` y `publicProfiles/{uid}` tras registrarse o iniciar sesión (`syncUserProfile`).

3. Colecciones y relaciones

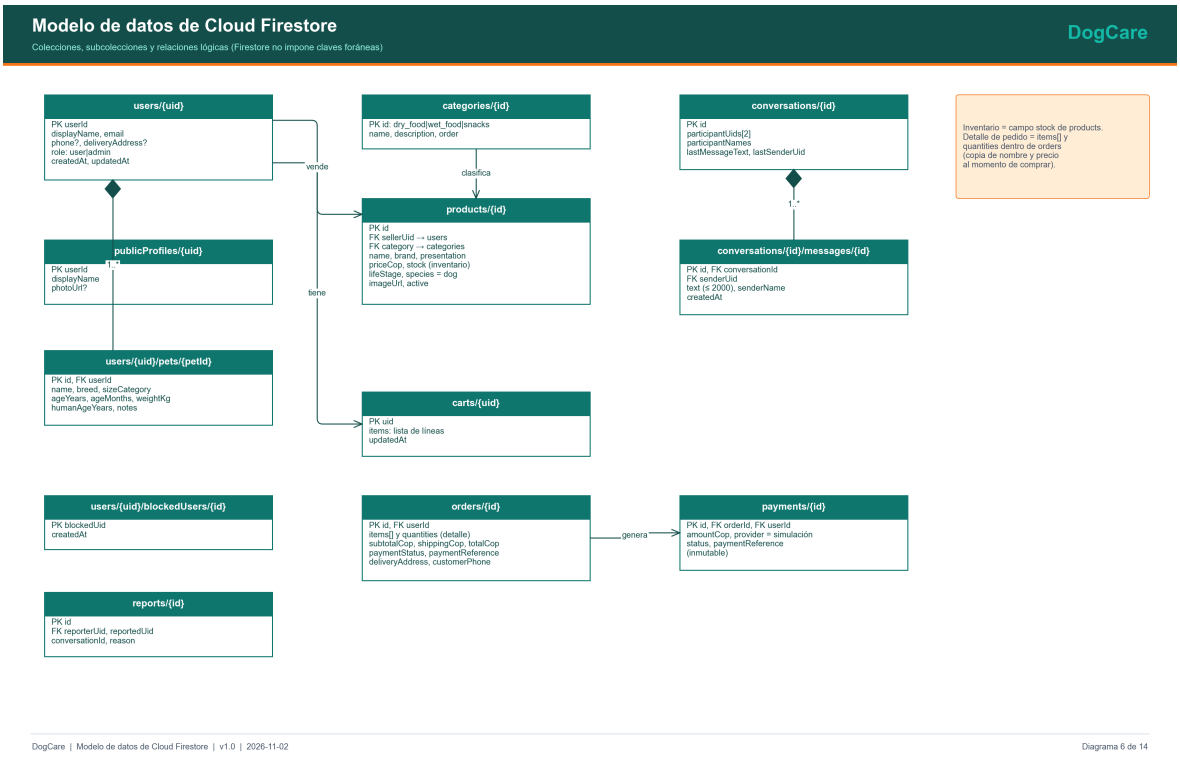


Figura 1. Modelo de datos.

Entidad de la guía	Ubicación	Justificación
Usuarios	`users/{uid}`, `publicProfiles/{uid}`	Perfil privado y público separados.
Perros	`users/{uid}/pets/{id}`	Subcolección: pertenece a un usuario.
Productos	`products/{id}`	`sellerUid` y `sellerName` por producto.
Categorías	`categories/{id}`	Tres fijas, sin escritura desde la app.
Inventario	`products.stock`	Un solo inventario por producto; evita duplicar y desincronizar.
Carrito	`carts/{uid}`	Un carrito por usuario.
Pedidos	`orders/{id}`	Cabecera.
Detalles de pedido	`orders.items[]`, `orders.quantities`	Copia de nombre y precio al comprar.
Pagos	`payments/{id}`	Comprobante inmutable.
Conversaciones	`conversations/{id}`	Dos participantes.
Mensajes	`conversations/{id}/messages/{id}`	Solo lectura y creación.

4. Reglas de seguridad (`firestore.rules`)

Colección	Lectura	Escritura
`users/{uid}`	Dueño o admin	Dueño; el rol no se asigna ni cambia desde la app
`users/{uid}/pets`	Dueño o admin	Dueño con validación
`users/{uid}/blockedUsers`	Dueño	Dueño; no puede bloquearse a sí mismo
`publicProfiles`	Usuarios autenticados	Dueño, solo campos públicos
`products`	Pública	Autenticado crea los suyos (`sellerUid`); solo el vendedor edita o borra; comprador descuenta stock ligado a su pedido PENDING→APPROVED
`categories`	Pública	Nadie desde la app
`carts/{uid}`	Dueño	Dueño
`orders`	Dueño o admin	Crea el dueño en PENDING; una única transición a estado final con `paymentReference`; no cambia `totalCop`, `items` ni `userId`
`payments`	Dueño o admin	Crear con monto y estado iguales al pedido; inmutable
`conversations`, `messages`	Participantes	Participantes; remitente = autenticado; rechaza si el otro lo bloqueó
`reports`	Solo admin	Cualquier autenticado crea el suyo

5. Pruebas de las reglas

`firestore.test.js` contiene 23 pruebas que se ejecutaron en el emulador de Firestore en una sesión previa (todas superadas, según `docs/REGISTRO_CAMBIOS.md`). No se volvieron a ejecutar en esta sesión. Las reglas se desplegaron en el proyecto Jarvis en esa sesión.

6. Hallazgos de seguridad sobre las reglas

ID	Prioridad	Hallazgo	Estado
S-01	P1	`orders` no valida que `totalCop` coincida con los precios de `products`; el pago simulado permite que el cliente fije el estado. Exige backend (Cloud Functions, plan de pago).	ACEPTADO Y DOCUMENTADO
S-02	P2	`conversations` permite actualizar `participantUids` mientras la lista conserve 2 elementos y al usuario: se podría cambiar al otro participante. Recomendación: exigir `incoming().participantUids == existing().participantUids`.	RECOMENDADO (requiere probar en emulador y desplegar)
S-03	P2	`isAdmin()` solo depende de `users/{uid}.role`; no hay custom claims. Cada evaluación suma lecturas.	DOCUMENTADO
S-04	P3	Productos antiguos sin `sellerUid` no se pueden borrar desde la app.	DOCUMENTADO

7. Integración y configuración

- `google-services.json` y `firebase_applet_config.xml` no se publican; hay plantillas en `config-examples/`.
- Registrar el SHA-1 de depuración y el token de App Check en la consola.
- Restringir la clave de API de Android por paquete y SHA-1.